

Componentes

Components

- Los componentes son la unidad de negocio más pequeña.
- Los componentes deben ser puros o “*PURE*”.
- Uncontrolled components and native HTML inputs.
- Component Library
 - Unstyled: Radix (React) and Headless UI
 - Styled: Shadcn / Material UI
 - Tailwindcss

Forms

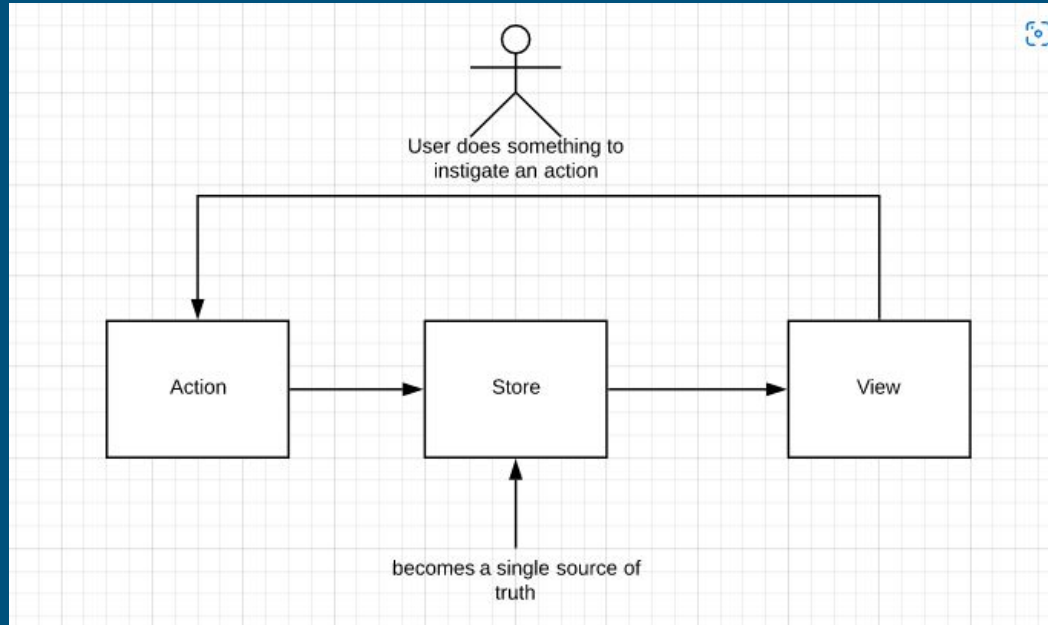
- Debemos hacer validaciones en el cliente.
- Podemos mostrar errores en tiempo real, sin necesidad ni de submitear la form ni de terminar de llenar el campo
- Podemos usar librerías externas de validación también
 - react-hook-forms
 - zod/yup

State management

"Backend del front" -nico

Unidirectional Data Flow

Buscamos que haya consistencia y un “Single source of truth”



Capa de estado

- Similar a persistencia de spring, esta capa contiene nuestras conexiones con **recursos asincrónicos** y nuestra **lógica de negocio**.
- Además contiene el **estado** en sí del cliente.
- Se utiliza para conectar y pasar información **entre componentes**.
- Stores
 - Separan el dato de la lógica de actualización
 - Nos obliga a ser más explicitos en como cambiamos el estado
 - Manejo nativo vía useState
 - Redux, Suztand, etc

RXJS - Observables

- [Angular - Using observables to pass values](#)
- Poderosa herramienta para manejar data asincrónica entre partes de la aplicación.
- Sigue un patrón similar a publish/subscribe
- Se utiliza principalmente en la librería HttpClient para obtener data de un API rest.
- Se puede utilizar para compartir data que eventualmente va a estar o que puede llegar a cambiar.

Subjects, BehaviourSubject

- [Angular - Observables in Angular](#)
- Async pipe
- Observables vs promises
 - [Angular - Observables compared to other techniques](#)
- Saving data as a Subject
 - [RxJS - Subject](#)
 - BehaviourSubject
 - One of the variants of Subjects is the BehaviorSubject, which has a notion of "the current value". It stores the latest value emitted to its consumers, and whenever a new Observer subscribes, it will immediately receive the "current value" from the BehaviorSubject.

Autenticación y Seguridad

La teoría

- Authorization header vs Cookies
 - Local Storage es vulnerable a ataques XSS.
 - Cookies son susceptibles a man-in-the-middle, en especial cuando TODOS los requests envían ambos tokens.
 - Las cookies nos dan menos control de cuando mandarse, sumando así el overhead de red, que impacta a todo, incluidos assets.
- Lógicas posibles
 - Todos los endpoints pueden loggear
 - /login nos devuelve en el body la información de loggeo

Manejo de errores

- Los manejos de errores se deberán hacer en el front
- Errores 404, 403

Interceptors

- [Angular - HttpInterceptor](#)
- Me permite interceptar globalmente todos los llamados y realizar acciones en el medio

Internacionalización

I18n

- i18next
- Pueden usar herramientas para extraer sus keys hacia un archivo
 - i18next scanner

```
...  
<div>{t('simpleContent')}</div>  
<Trans i18nKey="userMessagesUnread" count={count}>  
  Hello <strong title={t('nameTitle')}>{{name}}</strong>, you have {{count}} unread messa  
</Trans>  
...
```

Testing

Single Page Application

Testing

- Se puede testear distintas cosas en las SPA
 - Los componentes puros (UI Testing)
 - Tests de capa de estados
- Existen multiples herramientas de testinng
 - Jest / vitest
- Además necesitamos herramientas de Mocking
 - Nock / Sinon

Optimización

Single Page Application

Minificación js

- Example: [JavaScript Minifier \(javascript-minifier.com\)](https://javascript-minifier.com)
- Queremos optimizar recursos
- JS al ser ejecutado, es muy verborrágico, innecesariamente.
- A la vez ofuscamos levemente el código
- Técnicas
 - Acortamos variables
 - Obviamos ; cuando se puede
 - Optimizamos funciones
 - Eliminamos comentarios, breaklines, etc.

Minificación css

- Mismos fundamentos que la de js
- Técnicas
 - Eliminamos espacios
 - Renombramos variables
 - Usamos lefthand notation

Lighthouse

- Chrome's answer to slow internet
- Mejoremos la performance de las páginas web de manera replicable y subjetiva
- Hoy es común verlo en cualquier browser moderno
- Inclusive hay páginas que lo hacen online

SPA Hosting

Static hosting (Spring) / SSR

Spring

- Queremos hacer mvn package y que genere un paquete que tenga todo
- Vamos a usar un plugin que es el front end maven plugin
- github.com/eirslett/frontend-maven-plugin
 - Tiene todo: bower, npm, webpack, etc
 - Es el bridge entre el mundo maven y el mundo javascript
- Hay que tocar la configuración del proyecto webapp para que devuelva todos los recursos estáticos nuevos
- Hay que agregar la dependencia del módulo frontend para que se ejecute primero

Server side rendering

- Prepara un first impression que se baje MUY rápido
- Luego baja la página entera y la recarga sin que el usuario se de cuenta
- No vamos a poder utilizar nada en referencia a window en nuestro first impression
 - El código se PRE COMPILA en el servidor
 - No podemos usar por ejemplo localStorage, ya que no va a correr en el browser

Cache

Cache condicional y Cache busting

Cache

Cache condicional

- El request está siempre, por si cambio.
- Una vez que se hace el request incluyendo la fecha de cuando me lo descargue el servidor devuelve o no el archivo, dependiendo si hay uno nuevo (must-revalidate)

Cache no condicional

- No pido el recurso, con el header **expires** indicó por cuanto vale.
- Más eficiente
- Debe ser bien implementado para evitar clientes con recursos **stale**.

File revving

- Viene re revisión (o versioning)
- Para que el cache no condicional funcione necesitamos innovar
- El file revving se basa en técnicas de hashing, iterando los nombres de los archivos
- Iteramos los nombres de los archivos
 - Archivo Nuevo -> lo bajo

Implementación en Spring MVC

- Caching no condicional de recursos
- *registry.setCachePeriod()* en los resource handlers
- “cache-control” header en las responses
 - `response.setHeader("Cache-Control", "public, max-age="+ MAX_TIME +", immutable");`
- Se puede agregar un filtro en las configuraciones que haga esto para todas las responses (hay que implementar el filtro)

```
<!-- Unconditional Cache Filter -->
<filter>
  <filter-name>unconditionalCacheFilter</filter-name>
  <filter-class>ar.edu.itba.paw.webapp.config.UnconditionalCacheFilter</filter-class>
</filter>
```

- <https://www.baeldung.com/cachable-static-assets-with-spring-mvc>
- <https://www.baeldung.com/spring-mvc-cache-headers>